Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: *«Веб-программирование»*

Отчёт
По лабораторной работе №2
Вариант **13**

Выполнил:
Крамарь Михаил Романович
P3213

Проверил:
Бойко Владислав Алексеевич

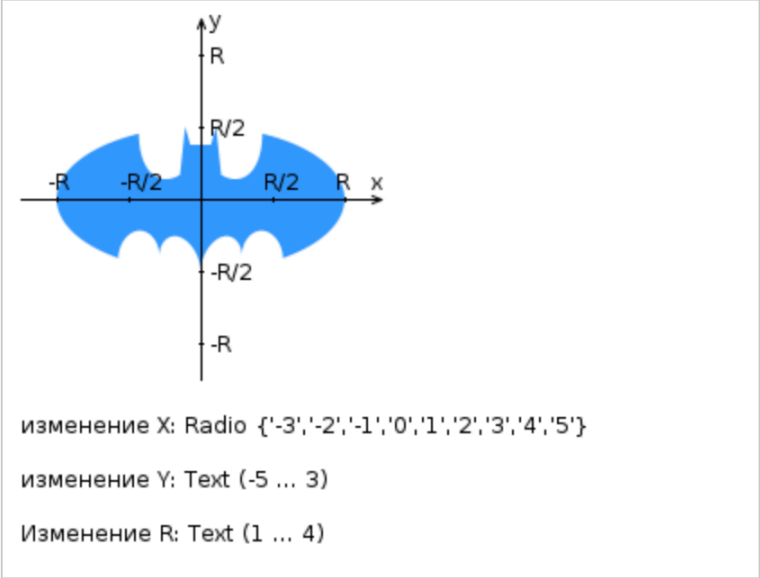
г. Санкт-Петербург
2026

Содержание

Текст задания 3
Ход работы 5
Ссылки на полезные ресурсы.....7
Вывод.....8

Текст задания

Доработать ЛР №1 и разработать FastCGI сервер на языке Java, определяющий попадание точки на координатной плоскости в заданную область, и создать HTML-страницу, которая формирует данные для отправки их на обработку этому серверу.



Параметр R и координаты точки должны передаваться серверу посредством HTTP-запроса. Сервер должен выполнять валидацию данных и возвращать HTML-страницу с таблицей, содержащей полученные параметры и результат вычислений — факт попадания или непопадания точки в область (допускается в ответе сервера возвращать json строку, вместо html-страницы). Предыдущие результаты должны сохраняться между запросами и отображаться в таблице.

Кроме того, ответ должен содержать данные о текущем времени и времени работы скрипта.

Комментарии по выполнению ЛР:

- Требуется поднять Apache httpd веб-сервер от лица своего пользователя на гелиосе (шаблон файла конфигурации доступен для скачивания наверху страницы)
- Веб-сервер должен заниматься обслуживанием статического контента (html, css, js) и перенаправлять запросы за динамическим контентом к FastCGI серверу
- FastCGI сервер требуется реализовать на языке Java (полезная библиотека в помощь в виде jar архива доступна для скачивания наверху страницы) и поднять также на гелиосе
- **Путем обращений из JavaScript к FastCGI серверу требуется показать понимание принципа AJAX**

Разработанная HTML-страница должна удовлетворять следующим требованиям (дополнительными, по отношению к ЛР №1, различаются по варианту):

- Данные формы должны передаваться на обработку посредством GET-запроса.

Ход работы

1. Изучение предметной области и документации

Первым делом я изучил текст задания и методические указания к лабораторной работе, чтобы понять, как должны взаимодействовать между собой клиентская часть, веб-сервер и FastCGI-приложение.

Я разобрал, чем синхронная обработка HTTP-запросов отличается от асинхронной, и изучил принцип работы AJAX: страница отправляет запрос в фоновом режиме и обновляет только нужную часть интерфейса, не перезагружаясь целиком. Для этого я использовал документацию MDN Web Docs по Fetch API и XMLHttpRequest. Отдельно я изучил технологии CGI и FastCGI.

2. Настройка веб-сервера Apache httpd

Далее я поднял веб-сервер Apache httpd от лица своего пользователя на сервере Helios. За основу я взял шаблон конфигурационного файла httpd.conf, в котором указал собственный порт, рабочие директории и пути к файлам журналов.

3. Разработка FastCGI-сервера на Java

Затем я реализовал серверную часть на Java. Для работы с протоколом FastCGI я подключил библиотеку в виде jar-архива. Основной цикл программы устроен следующим образом: сервер в бесконечном цикле вызывает FCGIaccept(), ожидая очередной запрос от веб-сервера, и обрабатывает его.

Поскольку FastCGI-процесс живёт между запросами, история предыдущих проверок хранится в памяти приложения (в списке результатов) и возвращается клиенту вместе с ответом на каждый запрос. При некорректных данных сервер отвечает кодом 400 Bad Request с описанием ошибочки.

4. Доработка клиентской части

После этого я доработал HTML-страницу из лабораторной работы №1.Запрос выполняется асинхронно средствами JavaScript (Fetch API). Полученный JSON-ответ разбирается на клиенте, а таблица результатов обновляется динамически без перезагрузки страницы. Таким образом, на практике реализован принцип AJAX. Кроме параметров и результата попадания, в таблице отображаются текущее время и время работы скрипта, полученные от сервера. Точка, соответствующая последнему запросу, дополнительно отрисовывается на Canvas.

Вся логика отрисовки области, перевода координат и работы с формой, реализованная в ЛР №1, была сохранена. Проверка попадания теперь выполняется на стороне сервера, а не в браузере. Была переделана область попадания по Бетмена😭😭😭.

5. Возникшие проблемы и их решение

Проглядел и дробные значения, содержащие запятую, некорректно разбирались сервером.

Я унифицировал формат на клиенте и на сервере: десятичным разделителем считается точка, а при получении значения с запятой оно приводится к стандартному виду перед разбором. Кроме того, на сервере разбор параметров обёрнут в обработку исключений, чтобы любое некорректное значение приводило к ответу 400, а не к падению приложения.

6. Запуск приложения

В завершение я развернул проект на сервере Helios: статические файлы (HTML, CSS, JS) разместил в каталоге, обслуживаемом Apache httpd, а FastCGI-приложение собрал в jar-архив и запустил на выделенном порту командой java -jar с подключённой библиотекой fastcgi-lib. В конце я конечно же всё проверил, всё работало, всё круто.

Ссылки на полезные ресурсы

Исходный код:

- https://github.com/moon2light-gif/ITMIUMIU/tree/main/3-4%E2%80%93WebProgramming/Lab2

Веб-сайт:

У Гелиоса все кастомные порты, к сожалению, закрытые

- Использованный мной ПОРТ := *50251*
- ssh -L <ПОРТ>:localhost:<ПОРТ> s502511@helios.cs.ifmo.ru -p 2222
- *http://localhost:<ПОРТ>*

Вывод

В ходе выполнения лабораторной работы №2 мной были получены навыки работы с веб-серверами и создания веб-приложений, а также улучшены знания и умения в области фронтенд-разработки на HTML, CSS и JavaScript; закреплены знания в области бэкенд-разработки на Java. Я подробно изучил работу веб-сервера Apache и разобрался как работает FCGI-сервер. В процессе я также закрепил свои знания об архитектуре клиент-сервер, подробно изучил протокол HTTP для передачи текстовых данных.